

# Introduction to Software Architecture

Lecturer: Ngo Huy Bien  
Software Engineering Department  
Faculty of Information Technology  
VNUHCM - University of Science  
Ho Chi Minh City, Vietnam  
[nhbien@fit.hcmus.edu.vn](mailto:nhbien@fit.hcmus.edu.vn)

## Objectives

- To explain *what* software architecture is.
- To explain *why* software architecture is important.
- To *communicate* in the field of software architecture.



## Contents

- I. Software Architecture
- II. Architectural Representation
- III. Architecture Design Method
- IV. Architecture Benefits
- V. Architecture Evaluation



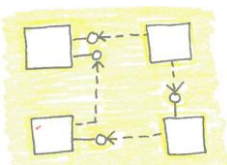
## References

1. Frank Buschmann et al. (1996). Pattern-Oriented Software Architecture: A System Of Patterns. John Wiley & Sons Ltd book.
2. David Garlan and Mary Shaw (1994). An Introduction to Software Architecture.
3. Klaus Pohl et al. (2005). Software Product Line Engineering - Foundations, Principles, and Techniques.
4. Len Bass, Paul Clements and Rick Kazman (2021). Software Architecture in Practice. Addison-Wesley.
5. Robert Cloutier et al. (2010). The Concept of Reference Architectures.
6. Philippe Kruchten (2009). Documentation of Software Architecture from a Knowledge Management Perspective - Design Representation.
7. Philippe Kruchten (1995). Architectural Blueprints - The 4+1 View Model of Software Architecture.
8. Craig Larman (2004). Applying UML And Patterns. 3rd Edition, Prentice Hall.
9. Nick Rozanski and Eoin Woods (2012). Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives. Addison-Wesley Professional.
10. Robert C. Martin (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson Education.



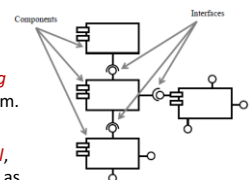
## What is Software Architecture? [1]

- A *software architecture* is a description of the subsystems and components of a software system and the relationships between them.
- The *software architecture* of a system is an artifact.
  - It is the result of the software design activity.
  - It is a solution to a problem.
  - Therefore we need a *problem* first.
    - *Functionality, quality attributes.*



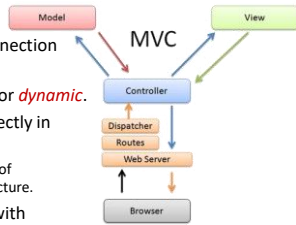
## Component

- A *component* is an encapsulated part of a software system.
- A component has an *interface*.
- Components serve as the *building blocks* for the structure of a system.
- At a *programming-language level*, components may be represented as modules, classes, objects or a set of related functions.



## Relationship

- A **relationship** denotes a connection between components.
- A relationship may be **static** or **dynamic**.
- Static relationships show directly in **source code**.
  - They deal with **the placement** of components within an architecture.
- Dynamic relationships deal with temporal connections and **dynamic interaction** between components.
  - They may **not be easily visible** from the static structure of source code.



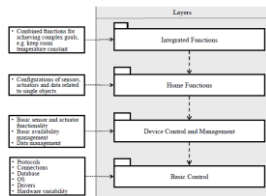
## What is Software Architecture? [2]

- An **architecture** of a specific system is a collection of computational components — or simply **components** — together with a description of the interactions between these components — the **connectors**.
- This leads to a view of an abstract architectural description as **a graph** in which the **nodes** represent the components and the **arcs** represent the connectors.
- **Connectors** can represent interactions as varied as **procedure call**, **event broadcast**, **database queries**, and **pipes**.

## What is Software Architecture? [3]

- The **architectural structure** is the **decomposition** of a software system into **parts** and **relationships**.
- The **architectural texture** is the collection of common **development rules** for realizing the system.

### Structure in Home Automation



## Texture in Home Automation

- **The use of layering** in the structure, as described in Structure.
- **The use of a hierarchy of layers**, subsystems, and components as described in Structure.
- **The facade pattern** that demands a single interface at subsystem level.
- **The observer pattern**, which decouples user interface issues from the data.
- **The presence of an initialization interface** with a prescribed set of functions at each component.
- **The use of high-priority processes for user interface** handling and **medium-priority processes** for user functions.

## What is Software Architecture? [4]

- The **software architecture** of a program or computing system is the structure or **structures of the system**, which comprise **software elements**, the externally visible properties of those elements, and the **relationships** among them.
  - An architecture is foremost an **abstraction** of a system that suppresses details of elements that do not affect how they use, are used by, relate to, or interact with other elements.
  - **Every computing system with software has a software architecture.**
  - An architecture can exist **independently** of its description or specification.
  - The **behavior** of each element is part of the architecture.
  - The definition is indifferent as to whether the architecture for a system is a **good** one or a **bad** one.

## Other Points of View

- Architecture is **high-level design**.
  - Other tasks associated with design are not architectural, such as deciding on important **data structures** that will be encapsulated.
- Architecture is the **overall structure** of the system.
  - The multiplicity of structures in an architecture lies at the heart of the concept.
- Architecture is the **structure of the components** of a program or system, their **interrelationships**, and the **principles** and **guidelines** governing their design and evolution over time.
  - Any system has an architecture that can be discovered and analyzed independently of any knowledge of the process by which the architecture was designed or evolved.
- Architecture is **components** and **connectors**.
  - Connectors imply a runtime mechanism.
  - When we speak of "relationships" among elements, we intend to capture both runtime and non-runtime relationships.

## Reference Models

- A **reference model** is a division of functionality together with data flow between the pieces.
- A reference model is a **standard decomposition** of a known problem into parts that cooperatively solve the problem.
- Arising from experience, reference models are a characteristic of **mature domains**.
  - Can you name the **standard parts** of a compiler or a database management system?
  - Can you explain in **broad terms** how the parts work together to accomplish their collective purpose?
  - If so, it is because you have been taught a **reference model** of these applications.

## Reference Architecture

- A **reference architecture** is a reference model mapped onto software elements (that cooperatively implement the functionality defined in the reference model) and the data flows between them.
- Whereas a **reference model** divides the functionality, a **reference architecture** is the mapping of that functionality onto a system decomposition.
- The mapping may be, but by no means necessarily is, **one to one**. A software element may implement part of a function or several functions.

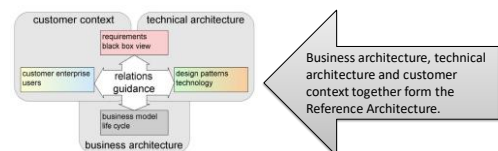
## What is Reference Architecture? [5]

- A **reference architecture** is an elaboration of company (or consortium) mission, vision, and strategy.
- Such reference architecture facilitates a **shared understanding** across multiple products, organizations, and disciplines about the current architecture and the vision on the future direction.

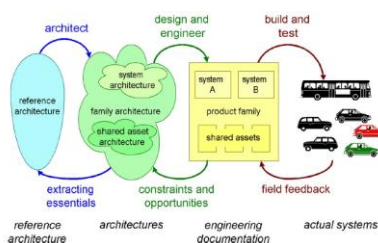


## Reference Architecture Elements

- Reference architecture is largely **based on** the explicit understanding of the relations between the business architecture, the technical architecture, and the customer context.
- In practice, **business architecture** and **customer context** are often missing. As a consequence, these technical Reference Architectures represent solutions for unspecified problems in unspecified contexts.

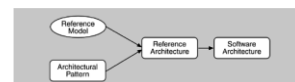


## Reference Architecture Abstraction Level



## Relationship [4]

- Reference models, architectural patterns, and reference architectures are not architectures; they are useful **concepts** that capture elements of an architecture.
- Each is the outcome of **early design decisions**.



The relationships of reference models, architectural patterns, reference architectures, and software architectures. (The arrows indicate that subsequent concepts contain more design elements.)



## Architectural Representation I: Boxes and Arrows [6]

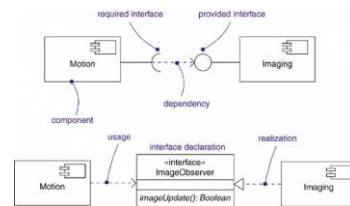
- Not much precise semantics behind the boxes and the arrows.
- Still very valuable in bridging the gaps between various groups of stakeholders.



## Architectural Representation II: Architecture Description Languages

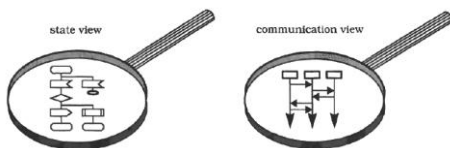
- What ADLs have in common is that they are able to
  - denote essential **elements** of an architecture – components, package,
  - as well as the **relationship** between these components: connectors, interfaces, and
  - to a certain extent some of the **characteristics** and **behavior** of these components and connectors –
  - so that some form of analysis, verification, and reasoning can occur, to derive or assess completeness, consistency, ambiguity, performance, and other qualities.
- ADLs often offer both a **textual** and a **graphical notation**, the intention being to have them readable by both humans and machines.

## Architectural Representation III: UML



## Model and View [1, 6]

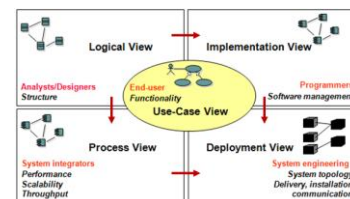
- A **model** is a **simplification** of reality, an **abstraction** of a system, created to better understand the system.
- A **view** represents a **partial aspect** of a software architecture that shows specific properties of a software system.
- A **view** is a **projection** of a **model**, which is seen from **one perspective** or vantage point and omits entities that are not relevant to this perspective.
- **Views** represent the interests of **different stakeholders**.



## The 4+1 View Model [7]

**Logical view** represents **functional** components and their relationships.

**Development view** focuses on **software** modules, packages, and environments used.

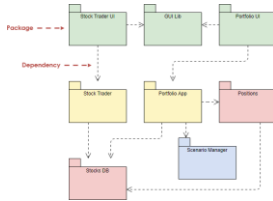


**Process view** shows a system's performance, scalability, workflow rules.

**Physical view** considers how software is mapped to hardware.

## The Logical View [8]

- The **logical architecture** is the large-scale organization of the software classes into packages (or namespaces), subsystems, and layers.
- This view is called **functional view**.
- It describes the **functional characteristics** of the software.



## The Development View

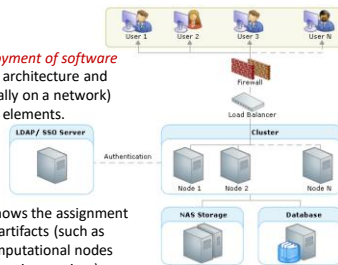
This view describes how the software is organized into **modules** or **compilation units** during the **development process**.



## The Deployment View

The software's static **organization** in its deployment environment.

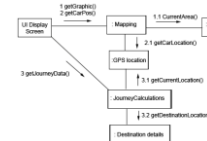
This view shows the **deployment of software elements** to the physical architecture and the communication (usually on a network) between physical elements.



A **deployment diagram** shows the assignment of concrete software artifacts (such as executable files) to computational nodes (something with processing services).

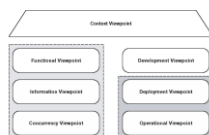
## The Process View

- The process architecture takes into account some non-functional requirements, such as **performance** and **availability**.
- It addresses issues of **concurrency** and **distribution**, of system's integrity, of fault-tolerance, and how the main abstractions from the logical view fit within the process architecture — on which thread of control is an operation for an object **actually executed**.
- At the highest level, the process architecture can be viewed as a set of independently executing logical networks of **communicating programs** (called "processes"), **distributed** across a set of hardware resources connected by a LAN or a WAN.



## Viewpoints [9]

- It would be hard work if every time you were **creating a view** of your architecture you had to go back to first principles to define **what should go into it**.
- A **viewpoint** is a collection of **patterns**, **templates**, and **conventions** for **constructing one type of view**. It defines the **stakeholders** whose concerns are reflected in the viewpoint and the guidelines, principles, and template models for constructing its views.



## Documenting Architecture: {Elements, Form, Rationale} [6]

- Architecture = {Elements, Form, Rationale}
- The **elements** are the main constituents of any architectural description in terms of **components** and **connectors**, while the **nonfunctional properties** guide the **final shape** or **form** of the architecture.
- There are several benefits of using **design rationales** in architecture as a mean to **explain why** a particular design choice is made or to know which **design alternatives** have been evaluated before the right or the optimal design choices are made.

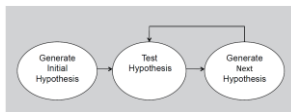
## Documenting Architecture: Architectural Knowledge

- **Architectural Knowledge** = Architectural Design + Architectural Design Decisions
  - Architectural Design
    - Viewpoints and Views
    - Architecture Description Languages
    - Application-Generic Knowledge: Patterns, Standards, Frameworks
  - Architectural Design Decisions
    - **Epitome** (or the Decision itself). This is a short textual statement of the design decision, a few words or a one liner. This text serves to summarize the decisions, to list them, to label them in diagrams.
    - **Rationale**. This is a textual explanation of the "why" of the decision, its justification. If the rationale is expressed in a complete external document, for example, a **tradeoff analysis**, then the rationale points to this document.
    - **Scope**. Some decisions may have limited scope, in time, in the organizations, or in the design and implementation.



## Architecture Design Method [4]

- **System Context Diagram, System Decomposition**
- Designing to **Architecturally Significant Requirements**
- **Generate and Test**



- **Where** does the initial hypothesis come from?
  - Styles, Patterns, Standards, and Frameworks



## Why Software Architecture?



Every computing system with software has an architecture.  
The code is the truth, but it is often too complicated.

- Simpler version of **resolution of a problem** (**modularization** and **information hiding**)
- A vehicle for **communication** among the system's stakeholders.
- Analysis of suitability of design **decisions**
- **Reusable** assets

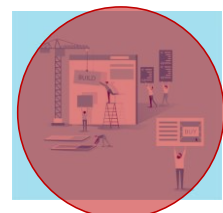


## Why Software Architecture?

- Your money, not that of investors.



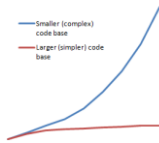
Cost Estimation



Buy vs. Build

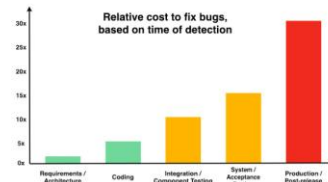
## But, Actually Why Software Architecture? [10]

- The *size* and *complexity* of software systems increase.
- The models are much *cheaper* than the real thing we are building.



Human lives

## The Cost of Wrong Architecture



Exponential cost of fixing bugs (Source: NIST).



## “Good” Architecture (I)

- Development time reduction**
  - Implementable (business cases support).
  - Understandability
    - Screaming (expressing its core business purpose)
  - Modular.
    - Independent UI. The UI can change easily.
    - Independent of any external agency.
    - Independent of the database and frameworks.
    - The business rules can be tested without the UI, database, web server, or any other external element.
- Deployment time reduction**
  - Units of deployment (DLLs, JARs, PHP files, gem files) are few.
  - Deployed with a single action.



## “Good” Architecture (II)

- Maintenance time reduction** (changes and extensibility effort reduction)
  - Independent of frameworks.
  - Independent of any external agency.
  - Independent of the database.
  - Plug-in architecture
  - Understandability
    - Screaming (expressing its core business purpose)
- (Optional) Operational feasibility
  - Scalability



## Evaluate an Architecture

- How to map *requirements* to components?
- How to *construct* each component? Which *technology* can be used to construct each component?
- How to organize, build and *integrate* components?
- How to *implement scenarios* based on the proposed components?
- How to *deploy, operate* and *replace* a component?
- How to *scale* a component?



Thank You & See You Again

